

Programação de Aplicações Web

Aula 8: Desenvolvimento de Software com Agentes de IA



A Mudança de Paradigma

Como evoluímos no desenvolvimento de software

A Evolução da Programação (Parte 1)

A forma como escrevemos código vem subindo o nível de abstração ao longo das décadas.

- **Código de Máquina e Assembly:** Onde tudo começou. O desenvolvedor precisava entender profundamente o hardware.
- **Linguagens de Alto Nível (C, Java):** Introduziram compiladores e sintaxe mais humana. O foco passou a ser na lógica e estrutura de dados.
- **Frameworks e Bibliotecas (React, Node.js):** Reutilização de código, abstraindo complexidades como manipulação direta de DOM e chamadas de rede.

O Padrão:

Em cada salto evolutivo, delegamos o "trabalho braçal" e repetitivo para focar mais na **resolução do problema** de negócio.

A Evolução da Programação (Parte 2)

O Novo Paradigma: Programação Guiada por Intenção

Hoje estamos vivendo mais um grande salto: a transição do "como fazer" para o **"o que eu quero"**.

Programação Tradicional

- Você escreve cada linha de código.
- Você define a lógica, cria os loops e as condições.
- Foco: "Como implementar a lógica X?"

Desenvolvimento com IA

- Você declara a intenção (prompt/especificação).
- A IA (ou agente) escreve a implementação.
- Foco: "Qual é a regra de negócio que quero atender?"

Desenvolvimento com IA e LLMs

O que é e como funciona

O que é o Desenvolvimento com IA?

Desenvolver com IA não significa que "o robô faz tudo e o programador perde o emprego". Pelo contrário, é uma parceria ("Pair Programming").

- A IA atua como um assistente supercapacitado (Copilot) ou um membro da equipe (Agente).
- O desenvolvedor assume o papel de **Arquiteto** e **Revisor**.
- As ferramentas auxiliam na criação de testes, refatoração de código, explicação de bugs e geração de *boilerplate* (código repetitivo).

Exemplo no Frontend: Em vez de criar o HTML, CSS e o componente React de um botão "do zero", você descreve: *"Crie um botão verde, arredondado, que dispara uma função de clique e tem estado de loading"*. A IA gera o código, e você revisa e integra.

O que são LLMs?

Para entender as ferramentas de IA, precisamos entender seu motor: os **LLMs (Large Language Models ou Modelos de Linguagem de Grande Escala)**.

- **Exemplos conhecidos:** GPT-5 (OpenAI), Claude 4.7 Sonnet (Anthropic), Gemini 3.1 Pro (Google).
- **Como funcionam?** São redes neurais gigantescas treinadas em imensas quantidades de texto (incluindo bilhões de linhas de código).
- **A Mágica:** Eles preveem a próxima palavra (ou token) com base no contexto anterior. Devido à sua escala, desenvolvem habilidades de raciocínio, lógica e interpretação de código incrivelmente avançadas.

Importante: LLMs não "copiam e colam" código de um banco de dados. Eles *geram* código novo, probabilístico, baseado nos padrões que aprenderam durante o treinamento.

Como os LLMs são usados no desenvolvimento?

Os LLMs atuam em diferentes etapas do fluxo de desenvolvimento:

1. **Autocomplete / Geração de Código:** Escrevem as próximas linhas enquanto você digita.
2. **Chat contextualizado:** Você pergunta sobre um arquivo ou um trecho de código específico (Ex: "Como otimizar este UseEffect?").
3. **Análise de Erros e Debugging:** Você cola o erro do terminal, e o LLM explica o que está acontecendo e como resolver.
4. **Geração de Testes Unitários:** O LLM analisa a função e cria todos os casos de teste (sucesso e falha).
5. **Automação Agêntica:** (Abordaremos na próxima seção).

A Era dos Agentes

Além dos Chatbots

O que são Agentes de IA?

Um simples chatbot (como a versão web do ChatGPT) recebe texto, processa e devolve texto. Ele está "preso" na tela de chat.

Um **Agente de IA** é um LLM equipado com a capacidade de **tomar decisões, usar ferramentas e agir** de forma autônoma (ou semi-autônoma) em um ambiente (como o seu sistema operacional ou sua IDE).



Percepção

O agente pode ler arquivos, ver logs do terminal e inspecionar a árvore do projeto.



Raciocínio

O agente analisa o problema, quebra em tarefas menores e planeja uma solução.



Ação

O agente pode usar ferramentas: criar, editar arquivos, executar comandos no terminal.



Feedback Loop

O agente analisa o resultado da ação (ex: falha num teste) e corrige o próprio erro.

Agentes vs Chatbots no Desenvolvimento

Com um Chatbot (ex: ChatGPT)	Com um Agente (ex: Antigravity)
1. Você copia o código do seu arquivo.	1. Você diz "Crie uma tela de login em <code>App.jsx</code> ".
2. Você cola no chat e pede para modificar.	2. O agente procura o arquivo sozinho, lê o conteúdo...
3. O Chat gera o código novo.	3. ...faz a edição diretamente no arquivo...
4. Você copia do chat e cola no seu editor.	4. ...roda o teste ou lint no terminal, e se falhar, ele mesmo arruma!

O Agente não apenas sugere o código, ele executa a tarefa dentro do seu ambiente.

Técnicas de Desenvolvimento

Como estruturamos o desenvolvimento (com e sem IA)

TDD e DDD: A Fundação

Antes de falar de desenvolvimento agêntico, precisamos lembrar como estruturamos o código.

TDD (Test-Driven Development)

Desenvolvimento Orientado a Testes. Você primeiro escreve o teste unitário (que inicialmente vai falhar), depois escreve o código mínimo para o teste passar, e por fim refatora.

- **Por que é útil com IA?** A IA é ótima em escrever código para fazer um teste passar! Isso garante que a IA não quebre regras de negócio (alucinação).

DDD (Domain-Driven Design)

Design Orientado a Domínio. É focar a modelagem do software nas regras do negócio ("O Domínio") e usar uma linguagem ubíqua (linguagem comum entre devs e especialistas do negócio).

- **Por que é útil com IA?** Modelos de linguagem entendem *muito* bem regras descritas em linguagem natural (humana). Quanto melhor seu DDD, melhor a IA vai codificar seu sistema.

SDD (Spec-Driven Development)

Com o avanço das ferramentas de IA, está surgindo o **Desenvolvimento Orientado a Especificação (SDD)**.

No SDD, a sua principal tarefa como desenvolvedor não é escrever o código em si, mas sim **escrever as regras e a especificação (o "Prompt")** de forma tão clara e detalhada que um Agente de IA possa implementar o código de forma determinística e precisa.

Em vez de focar no "Como":

```
function sum(a, b) {  
  return a + b;  
}
```

Focamos no "O Que":

```
# Funcionalidade: Carrinho  
Crie um componente de carrinho que:  
- Recebe uma lista de itens.  
- Soma o valor total (usando useMemo).  
- Mostre um botão "Finalizar" se total > 0.
```

Exemplo prático de fluxo com SDD

Como um desenvolvedor usa o SDD na prática no dia a dia:

1. **Definição da Tarefa:** O Dev abre o arquivo `tarefa-auth.md` e detalha em Markdown como a tela de Login deve funcionar, onde ficarão os arquivos, e as regras de negócio.
2. **Contexto:** O Dev orienta a IA para seguir a arquitetura definida (ex: "Siga o padrão da nossa aula 7, usando React Hooks").
3. **Execução:** O Dev pede ao **Agente de IA (ex: Antigravity)**: *"Implemente o que está descrito em tarefa-auth.md"*.
4. **Trabalho do Agente:** O agente lê o `.md` , cria o arquivo `Login.jsx` , adiciona o estilo, cria os testes e avisa: *"Finalizado"*.
5. **Revisão:** O Dev analisa o código gerado, testa a interface e aprova. Se algo estiver errado, em vez de alterar o código, ele **ajusta a especificação (o .md)** e pede para o agente refazer (evitando inconsistências entre código e documentação).

O Ecossistema de Ferramentas

Como incorporar IA no seu Workflow

IDEs Focadas em IA

Existem duas abordagens principais nas ferramentas atuais: **Extensões** para editores existentes e **IDEs Nativas** ("AI-First").

VS Code + Copilot

- **Categoria:** Extensão.
- O GitHub Copilot (baseado no OpenAI Codex) foi pioneiro no autocomplete avançado.
- Permite abrir um chat lateral para perguntar sobre o código.
- **Ponto Forte:** Você continua usando o VS Code que já conhece.

Cursor

- **Categoria:** IDE Nativa (Fork do VS Code).
- Construído do zero para integrar IA. Possui um recurso poderoso chamado "Composer" (Cmd+I) que pode editar múltiplos arquivos de uma vez.
- Excelente compreensão do contexto do projeto inteiro.
- **Ponto Forte:** Experiência incrivelmente fluida para SDD e refatoração em massa.

Ferramentas Agênticas: Antigravity e Claude Code

Enquanto as IDEs focam na "edição auxiliada", existem ferramentas projetadas para serem **Agentes Autônomos**.

Claude Code

- Agente via CLI (Linha de Comando) criado pela Anthropic.
- Você executa no terminal e ele interage com seu projeto.
- Excelente para tarefas em background, executar scripts, criar arquivos, fazer commits.

Antigravity

- Agente de IA avançado que atua diretamente no seu ambiente de desenvolvimento.
- Pode ler arquivos, buscar na web, rodar comandos de terminal e escrever código.
- **Diferencial:** Suporte a "Skills" (Habilidades personalizadas) e profunda integração para Pair Programming autônomo.
- *(Esta é a ferramenta que usaremos nas aulas práticas!)*

Como os Agentes entendem o seu Projeto?

Contexto e Protocolos

O Desafio do Contexto

Para que um Agente de IA consiga trabalhar no seu projeto, ele precisa de **Contexto**.

Se você pedir "Altere a cor do botão", o Agente precisa saber:

1. Em qual arquivo o botão está?
2. Nós usamos CSS puro, Tailwind ou Styled Components?
3. Qual é a paleta de cores oficial do projeto?

O maior desafio das ferramentas de IA é **como alimentar o LLM com o contexto correto**, já que as janelas de contexto (limite de palavras que o modelo consegue processar) são limitadas e cobrar muito contexto fica caro.

MCP (Model Context Protocol)

O **MCP (Model Context Protocol)** é um padrão aberto (open-source) criado para unificar como os modelos de IA se conectam a fontes de dados.

- **O Problema:** Antes, cada agente de IA tinha que ser programado especificamente para ler um banco Postgres, ou o GitHub, ou o Google Drive.
- **A Solução:** O MCP cria um padrão. Você instala um "Servidor MCP" (ex: MCP do Postgres) e qualquer Agente que suporte o padrão MCP consegue conversar com esse banco de dados.
- **Na prática:** Permite que o seu Agente (como o Cursor ou Antigravity) tenha acesso seguro a ferramentas externas locais e remotas para obter informações relevantes para o seu código.

ACP e AGENT.md

ACP (Agentic Coding Protocol)

Protocolos como o ACP definem como o agente interage com o editor de código e o sistema de arquivos. É o que permite que o agente não só veja o código, mas também aplique *diffs* (alterações), leia logs de erro em tempo real do terminal e entenda onde o cursor do programador está no momento.

AGENT.md (ou regras personalizadas)

É um arquivo que você coloca na raiz do seu projeto para "ensinar" o agente como ele deve se comportar **naquele repositório específico**.

```
# Exemplo de AGENT.md para o nosso curso
- Você é um desenvolvedor Frontend Sênior.
- Sempre utilize React com Functional Components e Hooks.
- Para estilização, utilize sempre Vanilla CSS (não use Tailwind).
- Sempre separe a lógica (JS) da apresentação (HTML) quando possível.
- Antes de commitar, sempre rode `npm run test`.
```

Desta forma, toda vez que o Antigravity ou Cursor for programar para você, ele vai ler essas regras antes de gerar qualquer linha de código!

Aprofundando no Antigravity

Skills e Fluxos Práticos

Antigravity e Skills

No **Antigravity**, além do Agente ter capacidades de IA, você pode ensinar "Skills" (Habilidades) a ele.

Uma **Skill** é um fluxo pré-programado ou uma macro que ensina o Agente a realizar uma tarefa complexa repetitiva com exatidão dentro do seu projeto.

Sem uma Skill:

Você pede: "Crie um componente de botão".

O Agente cria o componente, mas pode esquecer de criar o arquivo de teste ou colocar o arquivo na pasta errada (ex: em `/components` em vez de `/src/components/ui`).

Com uma Skill ("Create React Component"):

O agente possui uma regra clara:

1. Criar o arquivo em `/src/components`.
2. Criar automaticamente `index.css` na mesma pasta.
3. Criar `Component.test.js`.
4. Garantir que tudo está exportado corretamente.

Na nossa disciplina, o Antigravity será o assistente oficial de desenvolvimento em sala de aula.

O Desenvolvedor do Futuro

A IA não substitui o programador, ela eleva o nível da profissão. A codificação pesada será feita pela IA. Mas o que sobra para o humano?

1. **Arquitetura de Software:** Pensar no sistema como um todo.
2. **Definição de Requisitos (SDD):** Saber explicar o que precisa ser feito de forma clara.
3. **Revisão Crítica e Debugging:** Entender o código gerado para aprovar e garantir segurança.
4. **Resolução de Problemas Complexos:** A IA é boa no óbvio, o desenvolvedor brilhará nos casos atípicos e difíceis.

Programar não será mais sobre "digitar código", será sobre "orquestrar soluções".

Dúvidas?

Próxima etapa: Vamos abrir o Antigravity na prática e criar nosso primeiro projeto React utilizando o Spec-Driven Development!